

The W-2 screen, and the zero that wasn't there

Prepared for Michael Lyman · Greenway Marijuana · books-46 · slice A complete

What you can do now that you could not do yesterday

Open **Accounting → Form W-2 & W-3 (Annual)**. You will see every employee's W-2 laid out box by box, with a plain sentence under each box saying where that figure came from. Below the individual forms sits the W-3 — the single cover sheet that totals them all. And in the middle of the page, deliberately placed above the forms rather than buried at the bottom, is the one thing that actually matters: **a line-by-line comparison of your W-3 totals against the four Form 941s you filed during the year**.

That comparison is the whole slice. I want to spend most of this report explaining why, because if you understand only one thing about the W-2 it should be this next section.

Slice A is complete. The full test suite stands at **435 files and 10,685 tests, all passing**, with zero lint warnings and a clean type check.

The most important idea in this entire report

Form W-2 is not a tax return.

Form 941 and Form 940 are tax returns. They compute a number, and that number is a debt: you file it, you pay it. If you get one wrong, you paid the wrong amount of tax, and the repair is arithmetic plus interest.

The W-2 computes nothing and pays nothing. It is an **information return** — a report of figures that were already locked in by twenty-six pay runs that happened months ago. Nothing on it is a decision. Every box is a copy of something your year-to-date totals already knew in March.

That single structural fact cuts two ways, and both matter to you.

The good news: a W-2 engine cannot make a tax mistake, because it does not compute tax. It can only make a *transcription* mistake. So the correct way to build one is not as a calculator with lots of rules — it is as a **copier with lots of cross-checks**. That is why the engine behind this screen refuses far more often than it computes, and why every refusal names the exact column that disagreed.

The bad news, and this is the reason the whole slice exists: because the W-2 only re-reports figures, an error *in* it is almost never an error *in* it. It is a mistake made in March that nobody noticed, arriving in January with a deadline attached.

Think about what that means in practice. By the time your W-3 disagrees with your four 941s, those 941s are **filed**, the money is **paid**, and the tax year is **closed**. You cannot fix it by editing a box. You are now filing Form W-2c, Form W-3c, and Form 941-X — three corrective forms, plus amended state filings, for a mistake that would have taken four seconds to fix in March.

This is why the reconciliation is section 4 of the screen and not an appendix. It is why the IRS devotes an entire titled section of its instructions to it, quoted verbatim on your screen. **The comparison is worth more than the form generator**, and the layout says so.

Your two W-2 traps, and why they look exactly like bugs

These are not generic employer traps. They are yours specifically, because Greenway is an LLC that elected S-corporation treatment and you own 85% of it.

Trap 1 — Box 1 will be bigger than boxes 3 and 5 on your own W-2, and that is correct

If the company pays for your health insurance, that premium is **not** a tax-free fringe benefit the way it would be for an ordinary employee. Because you own more than 2%, it is **wages**, and it must appear in box 1 of your own W-2.

But it is generally carved out of Social Security and Medicare by §3121(a)(2)(B). So it goes **in** box 1 and **not** in boxes 3 and 5.

The result is a W-2 where box 1 is visibly larger than boxes 3 and 5. It looks like a bug. It reads like a bug. It is not a bug.

Here is why this is genuinely dangerous rather than merely confusing. A person — or a naive piece of software — "fixes" it by making the boxes agree. There are only two ways to do that, and both are wrong:

- Raise boxes 3 and 5 to match box 1, and you have **overpaid Social Security and Medicare** on money that was never subject to them.
- Lower box 1 to match, and you have **understated your income** — and worse, you have destroyed your own deduction, because the matching deduction on your personal return is only available *because the premium appeared on the W-2 first*.

That is the single most common S-corporation W-2 error in practice. So the screen does three things about it: it colours the difference **green, not gold and certainly not red**, because colouring a correct thing as a warning is exactly how you teach someone to "fix" it; it prints the sentence "*Do not make these boxes agree*"; and it quotes the two instructions side by side, because read apart they are misleading and only read together do they tell the truth.

One design detail I want to flag, because it is the kind of thing that separates a system you can trust from one you cannot. My first version of that badge read "*2% shareholder*" — it tried to label the person. The compiler rejected it, because there is no such field, and there **must not be**: nothing in a wage record knows who your shareholders are. Ownership lives in the ledger. Had that field existed, the screen would have been asserting a corporate-law fact from a payroll row. The badge now states only what payroll can actually observe — *box 1 exceeds boxes 3 and 5* — and lets the engine supply the legal reason underneath. **A screen should report what it measured and cite who explained it.**

Trap 2 — Box 17 must be blank, and blank is not the same as forgotten

Washington levies no personal income tax. Boxes 15 through 20 exist for state and local **income** tax. You have none to report. The trap is that your payroll *does* withhold Washington money — Paid Family and Medical Leave, and WA Cares. Those are real deductions from your employees' cheques. They are **not income tax**, and they belong in **box 14**, not box 17.

Put them in box 17 and you have told the IRS your employees paid a state income tax that does not exist, and invited them to claim a deduction they are not entitled to.

So on your screen, box 17 renders **grey and labelled "blank on purpose", with the reason attached**. This matters more than it sounds. A box that is *supposed* to be empty must never look like one somebody forgot, because those two situations demand opposite responses — one needs leaving alone and one needs filling in, and an empty grey cell tells you nothing about which you are looking at.

The deadline, and the extension that does not extend what you think

For tax year 2026, your W-2s and W-3 are due **1 February 2027**.

Not 31 January. The statutory date is 31 January, but in 2027 that falls on a **Sunday**, so it rolls to the Monday.

The screen does not have "2027-02-01" typed into it anywhere. It **computes** that date from the business-day and holiday calendar, then displays it. I did it that way for a specific reason: the same date is quoted in the IRS instructions stored in this system, and if one were typed and the other quoted, the two could silently drift apart in some future year. Computed, they cannot. There is now a test that fails if the calendar and the document ever disagree.

And here is the part people get wrong. You can request an extension of time to *file* with the Social Security Administration. That extension has **no effect whatsoever** on the date you must hand copies to your employees. They are two separate obligations with two separate penalties, and the instruction saying so is quoted on your screen because the confusion is so common.

What being late actually costs

The penalty is charged **per form**, and it is a staircase. These are the IRS's own per-form amounts for filings due after 31 December 2026, multiplied out for twelve employees:

When you get it right	Per form	Twelve forms
Within 30 days of the due date	\$60	\$720
31 days late, up to 1 August	\$130	\$1,560
After 1 August, or never	\$340	\$4,080

Intentional disregard	\$690 floor	\$8,280+, no maximum
-----------------------	-------------	----------------------

Now the part that nearly everybody misses. There is a **second penalty**, under §6722, for failing to furnish the copy to the *employee* — and it carries **the same amounts again**. The IRS can charge both for the same form. So a genuinely late filing season, twelve employees, both penalties:

	One penalty	Both (§6721 + §6722)
Fixed within 30 days	\$720	\$1,440
Fixed after 1 August	\$4,080	\$8,160

Eight thousand dollars. For paperwork. Not tax — the tax was already paid correctly through the year.

What to do with this: the actionable rule is that if you find a mistake in February, **fix it in February**. The staircase means waiting for a convenient moment is the single most expensive decision available to you on this form. And note the words attached to the last row: *intentional disregard* has **no maximum**. Everything else has a ceiling; deliberate non-compliance does not. Which is precisely why a system that stops you, tells you exactly what is wrong, and records that you fixed it has value beyond convenience — a documented process is the difference between a mistake and a decision.

One piece of good news: Greenway qualifies as a **small business** for these caps (average annual gross receipts of \$5 million or less over three years), which cuts the annual ceilings to roughly a third. At a dozen forms the ceilings are academic — you could never reach even the small-business cap. It is the **per-form** amount that will ever actually cost you money.

The arithmetic I let the IRS check for me

There is a nice detail in the instructions that I turned into a permanent test. The IRS says box 4 for 2026 "should not exceed \$11,439 (\$184,500 × 6.2%)."

They published the inputs *and* the answer. So that is not an illustration — it is an **oracle**, a sum where somebody else has already told me what the right answer is. In whole cents:

$$18,450,000 \times 6,200 \div 100,000 = \mathbf{1,143,900} - \textit{remainder zero}$$

It ties exactly, to the penny, with nothing left over. The engine must reproduce that figure or the test suite goes red. I am not trusting my own multiplication anywhere I can borrow somebody else's.

(A small subtlety worth knowing, since it looks like an inconsistency: the instruction says "*should not exceed*", not "*must not*". That is deliberate on their part — someone with two employers can legitimately have more than \$11,439 withheld across both. Per employer, per W-2, it is a hard ceiling; the excess gets recovered on the personal return.)

The same discipline applies to the reconciliation. Your four 941s carry **both halves** of Social Security and Medicare — yours and the employee's — while the W-3 carries only the employee's. So the 941 total should be about **twice** the W-3 figure. "About", not "exactly", and the reason is real: Additional Medicare Tax has **no employer match**, so for anyone paid over \$200,000 the true ratio dips slightly below two. An engine that demanded exactly double would raise a false alarm on a perfectly correct filing, and false alarms are how people learn to ignore alarms.

Now the part I think you will find most valuable: what building this screen found

You have said repeatedly that you want to know how the tool works and why to trust it. The honest answer is that you should trust it *because of what happens when it is attacked*, not because of what it looks like when it works. So here is what happened this week.

The zero that wasn't there

I ran the new test for the W-2 database reader for the first time. One assertion failed. It was not a test bug.

Somewhere in the code was a small function whose job was to read a wage figure out of the database and turn it into a number. It carried a comment — a good one, written by me in an earlier slice — explaining that if a wage column cannot be read, treating it as **zero** is the most dangerous possible answer, because zero is a *real* wage figure and nobody would ever question it.

And then the code did exactly that.

The reason is a piece of JavaScript trivia with real consequences. `Number("")` — converting an empty value into a number — does not fail. It returns **0**. The guard checked whether the *result* was a valid whole number instead of checking whether the *text going in* was actually digits. So:

What was in the column	What the old code produced
<code>(empty)</code>	0 — accepted as a real wage of zero
<code>(spaces)</code>	0 — accepted as a real wage of zero
<code>0x1F</code>	31
<code>1e3</code>	1,000
<code>900000</code>	900,000
<code>9007199254740993</code>	9007199254740992 — accepted, off by one, silently

Look at the last row, because it is the worse of the two defects. That number is not garbage — it is a perfectly good figure that is simply too large for JavaScript to hold exactly. The old check asked "is this a whole number?" and the answer was **yes**. It just wasn't the *same* whole number. It came back off by one, finite, integer-shaped, and wrong. A different test value came back off by two.

Both defects had this in common: **the comment named the hazard, and the code underneath did not guard it**. That is the most dangerous shape a defect can take, because the comment is what stops the next person looking.

It was not in one place. It was in six.

This is the part that made it worth a whole slice. I went looking for that same idiom elsewhere, and found it copied — with the same reassuring comment attached — into **six independent readers**: the W-2 reader, year-to-date totals, garnishments, W-4 onboarding, loans, and the ATM.

Six places where an unreadable wage column could become a silent zero. And every one of them was **untestable by construction**: each was a private helper inside a server-only file, which no unit test in this repository could reach. They were not neglected. They were *unreachable*.

The fix is one shared reader (`pg-bigint.ts` , 397 lines, 67 tests) that validates the **text** before converting it, and refuses anything that is not plain digits. All six now call it. The bad idiom has **zero occurrences** left in the source.

Two design choices inside it are worth your time:

I split it by nullability, not by a flag. There are two functions: one for columns the database guarantees are always filled, and one for columns that are genuinely allowed to be empty. I could have written one function with a switch. I did not, because a switch is something a caller can get wrong. And there is a test proving the two agree on every value that is actually present — so the only difference between them is the one difference that is supposed to exist.

"Empty" and "unreadable" are now different answers. The old nullable version returned "nothing here" for *garbage* as well as for *genuinely blank*. Those must never be the same answer. An empty column is a fact about your business. An unreadable column is a fact about a broken database, and it deserves to stop the page.

And then I checked that the new test could actually fail

A test that has never failed is not a test — it is decoration. So I put the broken version of the reader back and re-ran everything. **Five test failures and fifteen self-test failures**, each naming the exact defect. Then I restored the fix and confirmed all sixty-seven passing again.

That is the step that makes the difference. Anyone can write a green check mark.

Three more defects, all found by the same thing: making the code *reachable*

There is a pattern in this project you should know about, because it is going to keep recurring. Code that nothing uses looks perfect. It is only when something actually consumes it that the defects surface. Three examples from this week:

1. Seven dead links in a panel that already had a guard

Your W-2 screen shows 41 legal citations, each with a "Read the original" link. Those links come from two files.

The main file has a long note at the top explaining how it caught and fixed a bug where those links pointed at internal file paths instead of real web addresses — dead in a browser, and invisible to every automated check because the check only asked whether the field was *non-empty*, and an internal path is gloriously non-empty. It fixed its own 28 citations and wrote a test.

The test loops over its own 28. The screen renders 41. The other thirteen are borrowed from two neighbouring files, and seven of those still had internal paths.

So seven of the forty-one links on your screen were dead. The fix was correct; its **scope** was not. A fix narrower than the bug leaves the bug.

I found it by measuring rather than reading — a small probe over the actual list, which reported "*sources that are NOT web addresses: 7*" and named them. And the new test now loops over **what the screen loops over**, because anything narrower reproduces the original mistake in the test layer.

There is a subtle bonus test I added there too. Splitting the internal path from the public link creates a *new* way to fail that neither half can see alone: someone could update the stored document to the 2027 instructions while the public link still points at the 2026 PDF. Every test would pass, and you would be reading a different year's rules from the ones the quotes were checked against. Both now have to agree on the year.

2. The database refused honestly, and the screen rendered the refusal as blank space

This is my favourite, because the bug lived in the *handoff* rather than in either piece.

The reader is careful. When no 941 figures have been recorded, it refuses to run the comparison and returns "nothing" — and its comment explains exactly why it will not just compare against zeros: in a year with no wages, zero against zero comes back **all green**. A clean bill of health that checked nothing at all. Its comment ends "*the screen says so in gold*."

The screen said nothing. My first draft rendered an empty card — a heading over blank space.

Think about how that reads. Every other card on that page fills with green ticks when things are fine. So a blank one, on that page, reads as *nothing to report*. The engine's honest refusal became a silent one in the last six inches of the pipeline.

It now says, in gold: "**This check has not been run, and that is not the same as passing it**" — then names exactly what is missing and explains why zeros cannot be substituted in either direction.

3. A comment I wrote last slice was simply false, and the full suite caught it

When I moved the six readers onto the shared one, a test elsewhere failed — the W-4 reader. The failure was right and my code was wrong.

I had used the "column may be empty" version, justified by a comment I wrote claiming that an empty value meant "*the employee left Step 3 blank — itself a real W-4 answer*."

That sentence is plausible and it is false. I queried the live database: all four of those columns are defined **NOT NULL**. A blank Step 3 is stored as **zero**. Empty cannot occur — so my comment had invented a meaning for a state the database forbids. This is exactly why the standing rule is *verify, never reason*: I reasoned, and I was fluent and wrong.

The interesting part is what happened next. The strict reader throws an error and stops. Correct in general — and **wrong here**, and the full test suite is what told me so. The function that calls it deliberately isolates damage **per employee**: one bad W-4 names one person and everybody else still gets paid. Letting the error fly would have converted that into a whole-payroll outage whose message named the *column* but not the *person* — leaving you told that some figure somewhere is unreadable, with no way to find whose.

Strictness that destroys your ability to act on it is not strictness. It is a worse outage. So the refusal is caught and turned back into the per-employee refusal the caller is built around. Nothing is guessed, the row is still refused, and the employee is still named.

Then I measured that the result still discriminates properly, rather than assuming the change was safe: **7 of 7 legitimate values accepted and returned unchanged** (including the largest number that survives exactly, because boundaries are where off-by-one errors hide), **12 of 12 malformed values refused**, and two bad rows named while two good rows stayed usable. Zero is still preserved as zero — so the new strictness is not over-eager either.

One more, about my own work: a green tick that was checking nothing

I added the "Slice A — complete" section to the internal roadmap, including a table of counts: line numbers, test counts, and an

authority count written as a sum — *28 own + 13 borrowed = 41*. Then I ran the suite. All green.

That green tick was the problem. This project has a test whose entire job is to re-derive every countable claim in the roadmap from the actual code, so the document cannot quietly rot. It passed because it had **no idea my new section existed**. A table of numbers that nothing verifies, sitting in a document that describes itself as trusted, is the same defect as the six unreachable readers — a claim protected by nothing.

So I extended that test: eight new checks that re-derive the line counts, cross-foot the authority sum *by calling the same function the screen calls* rather than by adding 28 and 13 (adding them in the test would only prove the test can add), compute the deadline from the calendar, and confirm the honest gap below stays in writing.

Then I attacked all eight with deliberate mutations. **Seven died correctly. One survived** — and it was a good lesson. My check that all six readers still use the shared one was written as "does the file contain this path?" I repointed a reader at a *deliberately broken* path, `pg-bigint-XX`, and the test stayed **green**: the real path is a *prefix* of the broken one. Worse, all six files also mention that path in their **comments**, so the check would have passed on a reader that only talked about the shared code without using it.

The fix was one character — a closing quote mark, which terminates the path so a suffixed one no longer matches — plus anchoring it to a real import statement rather than to prose. Re-ran the mutation: dead. All five mutations now killed, and the file confirmed byte-identical to before the campaign.

I am telling you about a one-character bug in my own test because it is the clearest possible illustration of the discipline: **a gate you have never watched fail is not a gate**.

What is honestly not finished

I would rather you hear these from me than discover them.

Nothing writes the filed-941 figures yet. The table exists and the reconciliation reads it, but the four rows have to be keyed in from the returns you actually filed. This is deliberate, and I want to be clear about why, because it looks like a gap and it is partly a principle: **the entire value of that comparison is that the figures come from a different source than this software**. If the screen computed them itself, it would only ever be agreeing with itself — which is the one thing a cross-check may never do. Recording them is a typing job, not a calculation: take the four returns you filed and enter the totals as they appear on the paper you sent.

A small data-entry screen for those four rows is the next piece of work in this area, and it is now on the roadmap with a test that fails if anyone quietly drops it.

Year-to-date totals do not yet carry PFML and WA Cares withheld, which is what box 14 wants. Currently on the roadmap, not yet built.

Worked examples still cover 8 of 82 lessons. The screen says so honestly rather than implying it is finished.

The three ATM tables have never been type-checked. I found this while repointing that reader: every row there is cast in a way that switches the compiler off entirely, so no column in that file has ever been verified. That is a real finding and it is recorded, but fixing it properly is its own slice, and I was not going to half-do it inside a W-2 commit.

What I still need from you

Unchanged from last time, and the first one is genuinely time-critical:

1. **The twelve 2027 rates.** Your first payroll is 1 January 2027. Every rate-dependent screen will keep refusing — correctly — until they are on file.
2. **Your legal name as it appears on your Social Security card**, and the link between your employee record and your shareholder record. Your own W-2 cannot be produced without both.
3. **The 2027 health insurance premium figure**, which is trap 1 above in actual dollars.
4. **Your filed July DOR return** (for slice E), **Form 7203** (which gates slice B), **Form 2553 and the CP261 confirmation**, and the ending AAA balance.

Per the standing rule, none of these will be invented. The engines will keep refusing and naming the exact document until the real evidence arrives. That is not stubbornness — a statement built on an assumed number is not a draft, it is a false statement

that happens to balance.

The one-paragraph version

Your W-2 screen is live and reachable from the menu. It shows every box with its derivation, the W-3 totals, and — the point of the whole slice — a line-by-line comparison against the four 941s you actually filed, because a W-2 error is almost never an error in the W-2; it is a March mistake arriving in January when it costs three corrective forms instead of four seconds. Both of your S-corporation traps are implemented and explained on screen rather than merely documented. The deadline is computed from the calendar, not typed, so it cannot drift from the law that sets it. And building it turned up one defective idiom copied into six unreachable readers that could silently turn an unreadable wage into a wage of zero, four more defects in the handoffs between working parts, and a one-character hole in a test I had just written — every one of them found by attacking the system after it was already green, which is the only moment worth attacking it.